Answers of Tutorial 11

Exercise 1)

```java
public class LinearSearchCount {

    public static int linearSearchCount(int[] arr, int target) {
        int count = 0;

        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == target) {
                count++;
            }
        }
        return count;
    }

    Run | Debug
    public static void main(String[] args) {
        int[] array = {3, 5, 2, 5, 4, 5};
        int target = 5;

        int result = linearSearchCount(array, target);
        System.out.println("Occurrences of " + target + ": " + result);
    }
}
```

Exercise 2)

a) Why does binary search require a sorted array? What happens if the array is unsorted?

Ans: Binary searching is accomplished by continually splitting the search area in two and in each split, determining which of the two halves to continue searching in by comparing the center element against the search target. In cases when the source data is not sorted, it is not guaranteed that the data to the left of the centre element will be less than or equal to the centre element or that the data to the right of the centre element will be greater than or equal to the centre element. Thus binary searching may miss the search target, return incorrect results, fail to find an existing element. As a result binary searching will only work if the source data is sorted.

b)

```java
import java.util.Arrays;

public class SearchComparison {

    public static int linearSearch(int[] arr, int target) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == target) {
                return i;
            }
        }
        return -1;
    }

    public static int binarySearch(int[] arr, int target) {
        int low = 0, high = arr.length - 1;

        while (low <= high) {
            int mid = (low + high) / 2;

            if (arr[mid] == target)
                return mid;
            else if (arr[mid] < target)
                low = mid + 1;
            else
                high = mid - 1;
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] array = new int[1000];

        for (int i = 0; i < array.length; i++) {
            array[i] = (int) (Math.random() * 10000);
        }

        int target = array[500];

        long startTime = System.nanoTime();
        linearSearch(array, target);
        long endTime = System.nanoTime();
        System.out.println("Linear Search Time: " + (endTime - startTime) + " ns");

        Arrays.sort(array);

        startTime = System.nanoTime();
        binarySearch(array, target);
        endTime = System.nanoTime();
        System.out.println("Binary Search Time: " + (endTime - startTime) + " ns");
    }
}
```

Exercise 3)

```java
import java.util.HashMap;

public class FruitCount {

    public static void main(String[] args) {
        String[] fruits = {"apple", "banana", "apple", "orange", "banana"};

        HashMap<String, Integer> fruitCount = new HashMap<>();

        for (String fruit : fruits) {
            if (!fruitCount.containsKey(fruit)) {
                fruitCount.put(fruit, value: 1);
            } else {
                fruitCount.put(fruit, fruitCount.get(fruit) + 1);
            }
        }

        System.out.println(fruitCount);
    }
}
```